

REHOSPACE REAL ESTATE PLATFORM (RREP)

VOLUME II

TECHNICAL ARCHITECTURE & ENGINEERING SPECIFICATION

Document Version: 1.0

Product Name:

RehoSpace Real Estate Platform (RREP)

Document Type:

Technical Architecture & Engineering Specification

Audience:

- Solution Architects
- Software Architects
- Backend Developers
- Frontend Developers
- Mobile Developers
- DevOps Engineers
- Database Engineers
- QA Engineers
- Technical Leads

Table of Contents

PART A — Architecture

1. Introduction
2. Engineering Objectives
3. Architectural Principles
4. System Architecture
5. High-Level Architecture
6. Modular Architecture
7. Shared Services Architecture
8. Deployment Architecture

PART B — Technology Stack

- 9. Technology Strategy
- 10. Backend Technology
- 11. Frontend Technology
- 12. Mobile Technology
- 13. Database Technology
- 14. Caching Strategy
- 15. Search Engine Strategy
- 16. Object Storage
- 17. GIS & Mapping
- 18. AI Infrastructure

PART C — Database Architecture

- 19. Database Standards
- 20. Entity Relationships
- 21. Naming Conventions
- 22. Data Types
- 23. Indexing
- 24. Soft Deletes
- 25. Auditing
- 26. Partitioning Strategy

PART D — Backend Engineering

- 27. Laravel Architecture
- 28. Module Structure
- 29. Controllers
- 30. Services
- 31. Repositories
- 32. Policies
- 33. Events
- 34. Queues
- 35. Jobs
- 36. Notifications

PART E — API Architecture

- 37. REST Standards

- 38. Authentication
 - 39. Authorization
 - 40. API Versioning
 - 41. Error Handling
 - 42. Pagination
 - 43. Filtering
 - 44. Webhooks
-

PART F — Frontend Standards

- 45. UI Architecture
 - 46. Component Standards
 - 47. Dashboard Standards
 - 48. Forms
 - 49. Tables
 - 50. Responsive Design
-

PART G — Security

- 51. Identity Management
 - 52. RBAC
 - 53. Encryption
 - 54. Secrets Management
 - 55. Security Monitoring
 - 56. OWASP Compliance
-

PART H — DevOps

- 57. Git Strategy
 - 58. Docker
 - 59. CI/CD
 - 60. Monitoring
 - 61. Logging
 - 62. Backup
 - 63. Disaster Recovery
-

PART I — Quality

- 64. Coding Standards
- 65. Testing Strategy

- 66. Performance Testing
 - 67. Security Testing
 - 68. Documentation Standards
-

PART J — Appendices

- 69. Folder Structure
 - 70. Naming Standards
 - 71. Design Patterns
 - 72. Future Architecture
-

PART A

CHAPTER 1

Introduction

Purpose

The Technical Architecture & Engineering Specification defines the engineering architecture, technical standards, development guidelines, infrastructure strategy, and implementation approach for the RehoSpace Real Estate Platform (RREP).

This document provides the technical foundation required for software engineers, architects, DevOps teams, and quality assurance teams to implement, maintain, and evolve the platform in accordance with the business requirements defined in **Volume I – Business & Functional Product Specification**.

The document emphasizes maintainability, scalability, security, extensibility, and long-term product sustainability.

Scope

This specification governs all engineering activities related to RREP, including:

- Software architecture
- Backend development
- Frontend development
- Mobile applications

- Database architecture
 - APIs
 - Authentication & Authorization
 - Security
 - DevOps
 - Infrastructure
 - Quality Assurance
 - Deployment
 - Monitoring
 - Logging
 - Future extensibility
-

Intended Audience

This document is intended for:

- Solution Architects
 - Technical Architects
 - Backend Engineers
 - Frontend Engineers
 - Flutter Developers
 - Database Engineers
 - DevOps Engineers
 - QA Engineers
 - Technical Project Managers
-

Relationship to Other Volumes

This document complements:

- **Volume 0 – Product Vision & Strategy**, which defines the strategic direction of RREP.
 - **Volume I – Business & Functional Product Specification**, which defines the functional and business requirements.
 - **Volume III – Implementation, Deployment & Operations Guide**, which defines operational procedures after development.
-

CHAPTER 2

Engineering Objectives

The engineering architecture of RREP shall achieve the following objectives:

2.1 Scalability

The platform shall support growth in:

- Organizations
- Branches
- Users
- Properties
- Transactions
- Documents
- API requests
- Integrations

without requiring significant architectural redesign.

2.2 Maintainability

The system shall be organized into clearly defined modules with consistent coding standards, allowing engineers to modify or extend functionality with minimal impact on other components.

2.3 Reusability

Business logic shall be implemented in reusable services and components wherever practical, reducing duplication and simplifying future product development.

2.4 Security

Security controls shall be incorporated throughout the platform, including authentication, authorization, encryption, auditing, secure coding practices, and infrastructure hardening.

2.5 Performance

The platform shall deliver responsive user experiences through optimized database design, efficient APIs, caching, asynchronous processing, and appropriate infrastructure sizing.

2.6 Reliability

The platform shall prioritize data integrity, fault tolerance, and recoverability through transactional consistency, backups, monitoring, and disaster recovery planning.

2.7 Extensibility

The architecture shall enable future capabilities such as:

- New business modules
- Mobile applications
- AI enhancements
- Third-party integrations
- Public APIs
- Partner-developed extensions

without requiring major structural changes.

2.8 Testability

The system shall be designed to facilitate automated testing, including unit, integration, feature, and performance testing.

CHAPTER 3

Architectural Principles

The technical architecture of RREP shall adhere to the following guiding principles.

3.1 Modular Architecture

Each business capability shall be implemented as an independent module with clearly defined boundaries, responsibilities, and interfaces.

Modules shall minimize direct dependencies on one another and communicate through shared services, events, or documented APIs where appropriate.

3.2 Separation of Concerns

Presentation, business logic, data access, and infrastructure responsibilities shall remain distinct to improve readability, maintainability, and testability.

3.3 API-First Design

All business capabilities shall be exposed through well-documented RESTful APIs, enabling consistent access from web clients, mobile applications, and third-party integrations.

3.4 Convention Over Configuration

Where practical, the platform shall follow established framework conventions to reduce complexity, improve developer productivity, and simplify onboarding.

3.5 Secure by Design

Security considerations shall be embedded throughout the architecture rather than treated as an afterthought.

This includes:

- Strong authentication
 - Fine-grained authorization
 - Secure communications
 - Data encryption
 - Audit logging
 - Secure coding practices
-

3.6 Domain-Oriented Design

The architecture shall reflect real-world business domains, ensuring that code organization aligns with business concepts and promotes long-term maintainability.

3.7 Event-Driven Capabilities

Business events such as completed sales, received payments, customer registrations, and workflow approvals shall publish domain events that can trigger notifications, integrations, analytics, or other asynchronous processes.

3.8 Cloud-Ready Design

Although RREP supports cloud, on-premises, and hybrid deployments, all components shall be designed with cloud-native principles where appropriate to maximize scalability and operational flexibility.

3.9 Technology Independence

Business logic should remain as independent as possible from infrastructure-specific implementations, allowing components to evolve without widespread changes.

CHAPTER 4 — SYSTEM ARCHITECTURE

4.1 Architectural Overview

RREP shall be implemented as a modular, API-first enterprise application consisting of a web application, optional mobile applications, backend business services, shared platform services, persistent data stores, external integrations, and an intelligence layer.

The architecture shall separate:

1. Presentation
2. Application/business logic
3. Domain modules
4. Infrastructure services
5. Persistent storage
6. External integrations
7. Artificial intelligence services

The architecture shall allow the product to be deployed as a standalone installation for an individual customer organization.

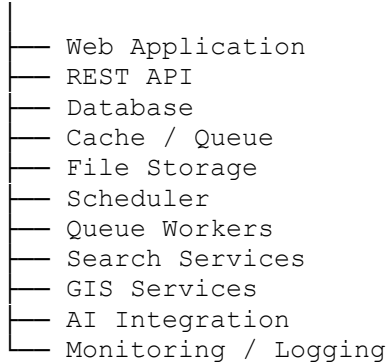
RREP is a reusable commercial product. It shall NOT be architected as a central multi-tenant SaaS environment in which RehoSpace manages multiple customer organizations inside one shared operational database.

Each customer deployment shall represent an independent installation/license of the platform.

4.2 Deployment Model

A customer installation shall logically consist of:

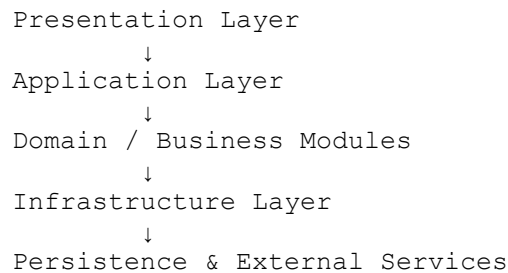
Customer RREP Installation



Each customer's operational data shall remain isolated within that customer's deployment.

4.3 Architectural Layers

RREP shall follow the following conceptual layers:



The presentation layer shall not directly contain business rules.

Controllers shall not become repositories for business logic.

Business logic shall be implemented in appropriate services, actions, domain objects, policies, events, and supporting components.

CHAPTER 5 — HIGH-LEVEL ARCHITECTURE

5.1 Logical Architecture

The system shall consist of the following major components:

Client Applications

- Administrative Web Application
- Staff Web Application
- Customer Portal
- Property Owner Portal
- Public Property Website
- Mobile Applications
- External API Consumers

Application Layer

- Authentication
- Authorization
- Business Modules
- Workflow Engine
- Notification Engine
- Reporting Engine
- Integration Engine
- AI Engine

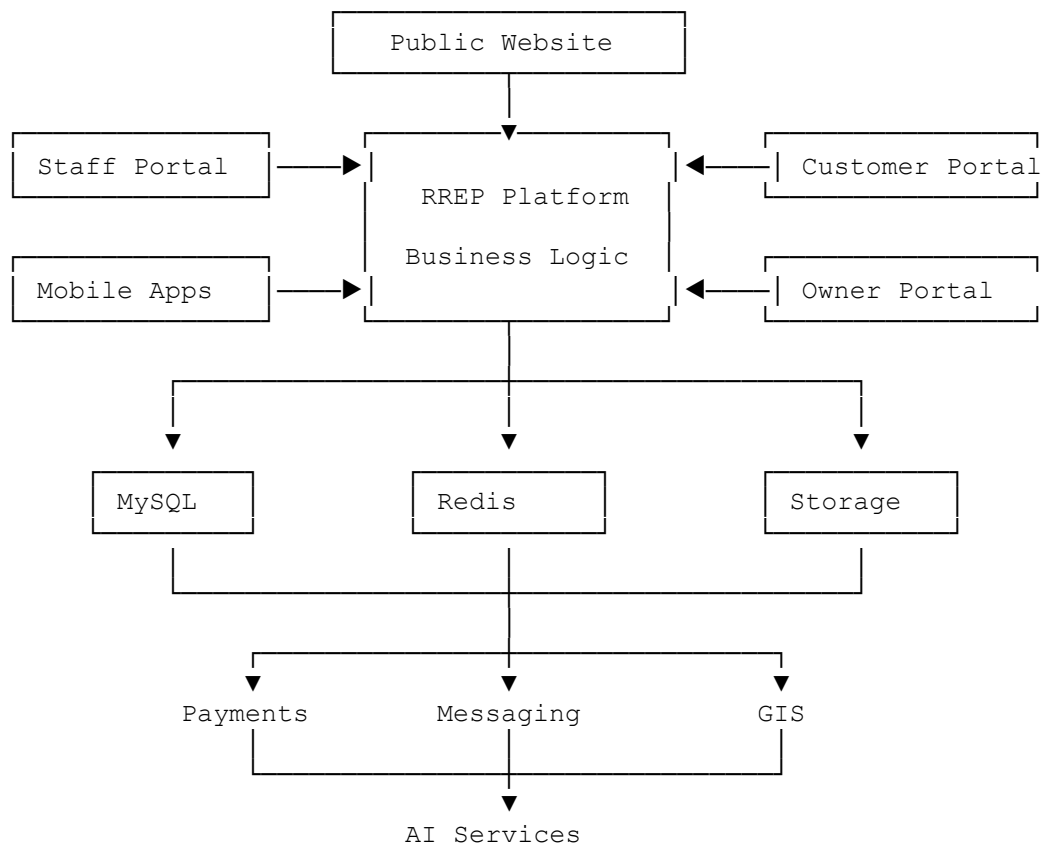
Infrastructure Layer

- Database
- Cache
- Queue
- File Storage
- Search
- Scheduler
- Logging
- Monitoring

External Services

- Payment gateways
 - SMS providers
 - Email providers
 - WhatsApp providers
 - Maps/GIS
 - AI providers
 - Government integrations
 - Other third-party APIs
-

5.2 Conceptual Diagram



CHAPTER 6 — MODULAR ARCHITECTURE

6.1 Principle

Each business capability shall be represented as a distinct application module.

Modules shall have:

- Clear responsibilities
 - Defined interfaces
 - Defined dependencies
 - Independent authorization rules
 - Module-specific services
 - Module-specific tests
 - Module-specific configuration
-

6.2 Core Modules

The implementation shall provide modules corresponding to the approved Product Specification.

At minimum:

- Platform Administration
- Organization & Branch Management
- User & Identity Management
- Property Management
- Land & Parcel Management
- Survey Management
- Customer & CRM
- Property Owner Management
- Sales
- Reservations
- Rentals & Lease Management
- Finance
- Commission Management
- Marketing
- Communication
- Document Management
- Workflow Management
- Maintenance
- Customer Portal
- Owner Portal
- Reporting & Business Intelligence
- Integration Management
- Audit & Compliance
- AI & Smart Automation

6.3 Module Independence

A module shall not directly manipulate another module's database tables unless a clearly documented architectural exception exists.

Preferred communication mechanisms are:

1. Service interfaces
 2. Domain events
 3. Application services
 4. APIs
-

6.4 Shared Kernel

Common functionality shall be implemented in a shared platform layer.

Examples:

- Authentication
- Users
- Roles
- Permissions
- Notifications
- Files
- Audit
- Settings
- Localization
- Workflow
- Number generation
- Activity logging

CHAPTER 7 — SHARED SERVICES ARCHITECTURE

RREP shall provide reusable platform services.

7.1 Authentication Service

Responsible for:

- Login
- Logout
- Password management
- Password reset
- Session management
- Multi-factor authentication
- API authentication

7.2 Authorization Service

Responsible for:

- Roles
- Permissions

- Policies
 - Access scopes
 - Branch restrictions
 - Department restrictions
 - Resource-level authorization
-

7.3 Notification Service

A centralized notification service shall support:

- In-app notifications
- Email
- SMS
- Push notifications
- WhatsApp where integrated

Modules shall publish notification requests rather than implementing provider-specific logic themselves.

7.4 File Service

A centralized file abstraction shall support:

- Upload
 - Download
 - Metadata
 - File categorization
 - Access control
 - Versioning
 - Storage provider abstraction
-

7.5 Audit Service

All critical modules shall use the centralized audit service.

7.6 Workflow Service

Business modules shall be able to register workflows containing:

- States

- Transitions
 - Approvals
 - Conditions
 - Actions
 - Notifications
-

7.7 Numbering Service

The platform shall support configurable numbering schemes for:

- Customers
- Properties
- Reservations
- Invoices
- Receipts
- Contracts
- Payments
- Survey records
- Documents

Number formats shall be configurable by organization.

CHAPTER 8 — DEPLOYMENT ARCHITECTURE

RREP shall support three principal deployment models.

8.1 Cloud Deployment

The application may be hosted on:

- VPS
 - Cloud VM
 - Managed cloud infrastructure
 - Containerized infrastructure
 - Kubernetes environments where justified
-

8.2 On-Premises Deployment

The platform may be installed within the customer's infrastructure.

Requirements shall include:

- Linux server support
 - Web server
 - PHP runtime
 - MySQL-compatible database
 - Queue worker
 - File storage
 - Scheduled jobs
-

8.3 Hybrid Deployment

The core platform may remain within the customer's environment while selected services operate externally.

Examples:

- Cloud AI
 - SMS
 - Email
 - Payment gateway
 - External GIS
-

PART B — TECHNOLOGY STACK

CHAPTER 9 — TECHNOLOGY STRATEGY

The technology stack shall prioritize:

- Long-term maintainability
- Developer availability
- Open standards
- Security
- Performance
- Cost efficiency
- Portability
- Community support

Technology decisions shall avoid unnecessary vendor lock-in.

CHAPTER 10 — BACKEND TECHNOLOGY

10.1 Framework

The backend shall use:

Laravel 11+ / current supported Laravel LTS-compatible version at implementation time.

The project shall follow Laravel conventions while maintaining a modular domain structure.

10.2 PHP

The project shall use a currently supported PHP version compatible with the selected Laravel version.

PHP version requirements shall be documented in deployment configuration.

10.3 Backend Responsibilities

The backend shall provide:

- Business logic
 - Authentication
 - Authorization
 - API
 - Database access
 - Validation
 - Workflows
 - Notifications
 - Queues
 - Events
 - Reporting
 - Integrations
 - AI orchestration
-

CHAPTER 11 — FRONTEND TECHNOLOGY

The primary administrative web interface shall use:

- Laravel Blade
- Bootstrap 5
- JavaScript
- jQuery
- Select2
- DataTables
- AJAX/fetch-based asynchronous operations where appropriate

The project shall NOT use Tailwind CSS, Alpine.js, or Livewire as mandatory architectural dependencies.

11.1 UI Principles

Interfaces shall be:

- Responsive
 - Accessible
 - Consistent
 - Component-based
 - Mobile-friendly
 - Performance-conscious
-

11.2 Frontend Components

Reusable components shall be created for:

- Buttons
- Modals
- Forms
- Tables
- Cards
- Alerts
- Filters
- Dropdowns
- Tabs
- Pagination
- File uploads
- Dashboards

- Charts
-

CHAPTER 12 — MOBILE TECHNOLOGY

Where mobile applications are required, the preferred approach shall be a cross-platform framework such as Flutter.

Mobile applications shall communicate with the platform through versioned APIs.

Potential applications include:

- Customer App
- Agent App
- Property Owner App
- Surveyor/Field App
- Management App

Mobile clients shall not directly access the database.

CHAPTER 13 — DATABASE TECHNOLOGY

13.1 Primary Database

RREP shall use MySQL as the primary relational database.

The implementation shall use a currently supported MySQL release compatible with the selected Laravel version.

13.2 Database Responsibilities

The relational database shall store:

- Users
- Properties
- Customers
- Transactions
- Financial records

- Workflows
 - Audit records
 - Configuration
 - Relationships
 - Operational metadata
-

13.3 Transactions

Financially and operationally critical operations shall use database transactions.

Examples:

- Sale completion
 - Payment posting
 - Refund
 - Reservation confirmation
 - Commission calculation
 - Allocation
-

CHAPTER 14 — CACHING STRATEGY

Redis shall be supported as the preferred cache and queue backend where infrastructure permits.

Caching shall be applied to:

- Configuration
- Permissions
- Reference data
- Expensive queries
- Dashboard aggregates
- API responses where appropriate

Caching shall never compromise data correctness.

CHAPTER 15 — SEARCH ENGINE STRATEGY

The platform shall initially support database-based search.

For installations requiring advanced search, a dedicated search engine may be introduced.

Potential capabilities:

- Full-text property search
- Location search
- Property filters
- Customer search
- Document search
- Natural language search

The search abstraction shall allow the implementation to change search providers without redesigning business modules.

CHAPTER 16 — OBJECT STORAGE

The system shall abstract file storage.

Supported implementations may include:

- Local storage
- S3-compatible object storage
- Cloud storage

Documents shall never be stored directly inside relational database fields as binary blobs unless specifically justified.

The database shall store file metadata and references.

CHAPTER 17 — GIS & MAPPING

The GIS architecture shall support:

- Coordinates
- Property locations
- Parcels
- Boundaries
- Points of interest
- Map visualization
- Spatial relationships
- Survey information

Potential providers include:

- OpenStreetMap
- Mapbox
- Google Maps
- Other GIS providers

The GIS provider shall be abstracted where practical.

CHAPTER 18 — AI INFRASTRUCTURE

AI shall be implemented through an abstraction layer rather than tightly coupling business modules to one AI provider.

The AI layer shall support:

- LLM providers
- OCR services
- Embedding services
- Computer vision
- Predictive models

Business modules shall request AI capabilities through internal AI services.

This permits future provider changes.

PART C — DATABASE ARCHITECTURE

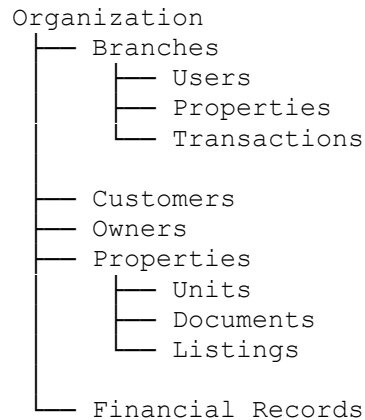
CHAPTER 19 — DATABASE STANDARDS

Database design shall prioritize:

- Data integrity
 - Normalization
 - Appropriate denormalization for reporting
 - Referential integrity
 - Index efficiency
 - Auditability
 - Scalability
-

CHAPTER 20 — ENTITY RELATIONSHIPS

Major entities shall include relationships such as:



Actual implementation relationships shall be documented through entity relationship diagrams in the engineering repository.

CHAPTER 21 — NAMING CONVENTIONS

Database tables shall use:

- lowercase
- snake_case
- plural nouns

Examples:

```
users
properties
property_units
sales_orders
payment_transactions
customer_documents
```

Primary keys shall use a consistent convention.

Foreign keys shall use:

```
<singular_entity>_id
```

Example:

property_id
customer_id
branch_id

CHAPTER 22 — DATA TYPES

Appropriate native database types shall be used.

Examples:

- BIGINT for high-volume identifiers where appropriate
- DECIMAL for monetary values
- DATETIME/TIMESTAMP for timestamps
- BOOLEAN/TINYINT for flags
- JSON only for genuinely flexible structured data

Monetary values shall never use floating-point types.

CHAPTER 23 — INDEXING

Indexes shall be created for:

- Foreign keys
- Frequently queried fields
- Unique identifiers
- Search fields
- Status fields where useful
- Date ranges
- Composite query patterns

Indexes shall be based on actual query behavior rather than indiscriminately indexing every field.

CHAPTER 24 — SOFT DELETES

Business records that require historical preservation shall use soft deletion where appropriate.

Examples:

- Properties
- Customers
- Documents

- Users
- Listings

Financial records and audit records shall generally not be physically deleted through ordinary application operations.

CHAPTER 25 — AUDITING

Important entities shall maintain:

- Created by
- Updated by
- Created at
- Updated at

Critical changes shall additionally be captured in the centralized audit system.

Audit data shall record:

- Actor
- Action
- Entity
- Previous state where appropriate
- New state where appropriate
- Timestamp
- IP
- Request/correlation identifier

CHAPTER 26 — PARTITIONING STRATEGY

Partitioning shall not be introduced prematurely.

It may be considered for high-volume tables such as:

- Audit logs
- Activity logs
- Notifications
- API logs
- Transaction histories

Partitioning decisions shall be based on measured production workload.

PART D — BACKEND ENGINEERING

CHAPTER 27 — LARAVEL ARCHITECTURE

RREP shall use Laravel as the primary backend framework.

The application shall follow a layered architecture:

```
HTTP Layer
↓
Application Services
↓
Domain Modules
↓
Infrastructure
↓
Persistence / External Services
```

CHAPTER 28 — MODULE STRUCTURE

The recommended structure shall organize code by domain.

Example:

```
app/
├── Modules/
│   ├── Property/
│   ├── CRM/
│   ├── Sales/
│   ├── Finance/
│   ├── Survey/
│   ├── Marketing/
│   ├── Documents/
│   ├── Workflow/
│   ├── Reporting/
│   ├── Integrations/
│   └── AI/
├── Shared/
│   ├── Services/
│   ├── Support/
│   ├── DTOs/
│   ├── Events/
│   └── Exceptions/
└── Infrastructure/
```

The exact structure may be refined during implementation while preserving modular boundaries.

CHAPTER 29 — CONTROLLERS

Controllers shall:

- Receive requests
- Validate input
- Authorize access
- Invoke application services
- Return responses

Controllers shall NOT contain extensive business logic.

A controller should generally orchestrate rather than implement complex business rules.

CHAPTER 30 — SERVICES

Services shall contain reusable business operations.

Examples:

```
PropertyService  
SaleService  
ReservationService  
PaymentService  
CommissionService  
DocumentService  
NotificationService  
AIService
```

Services shall be testable independently of HTTP controllers.

CHAPTER 31 — REPOSITORIES

Repositories may be used where they provide genuine abstraction value.

Repositories shall not be introduced merely for the sake of architectural ceremony.

They are appropriate when:

- Complex data access requires abstraction.
- Multiple storage implementations may exist.
- Query logic is extensive.
- Testing benefits from isolation.

Simple Laravel Eloquent operations may remain within appropriate application services.

CHAPTER 32 — POLICIES

Laravel Policies shall enforce resource-level authorization.

Examples:

```
PropertyPolicy
CustomerPolicy
SalePolicy
PaymentPolicy
DocumentPolicy
```

Authorization shall be enforced server-side regardless of frontend visibility.

CHAPTER 33 — EVENTS

Domain events shall be used for significant business occurrences.

Examples:

```
PropertyCreated
PropertyPublished
ReservationCreated
ReservationApproved
SaleCompleted
PaymentReceived
LeaseExpired
DocumentUploaded
WorkflowApproved
```

Events may trigger:

- Notifications
 - Audit records
 - Integrations
 - Analytics
 - AI processing
-

CHAPTER 34 — QUEUES

Long-running operations shall be processed asynchronously.

Examples:

- Email delivery
- SMS
- WhatsApp
- Report generation
- Document processing
- OCR
- AI requests
- Large imports
- Exports
- Image processing

Redis may be used as the queue backend.

CHAPTER 35 — JOBS

Jobs shall be:

- Idempotent where possible
- Retryable
- Observable
- Appropriately timed out
- Logged

Failed jobs shall be visible to administrators.

CHAPTER 36 — NOTIFICATIONS

Notification templates shall be centralized.

Templates shall support:

- Variables
- Localization
- Channel-specific formatting
- Scheduling
- Delivery status

Notification providers shall be abstracted behind provider services.

PART E — API ARCHITECTURE

CHAPTER 37 — REST STANDARDS

RREP APIs shall follow REST principles.

Example:

```
GET    /api/v1/properties
POST   /api/v1/properties
GET    /api/v1/properties/{id}
PUT    /api/v1/properties/{id}
DELETE /api/v1/properties/{id}
```

Endpoints shall use nouns rather than action-heavy URLs wherever practical.

CHAPTER 38 — AUTHENTICATION

API authentication shall support secure token-based mechanisms.

The exact implementation shall be selected according to client requirements.

The API shall support:

- Token authentication
 - Token expiration/revocation
 - Password authentication where applicable
 - MFA for privileged users
 - OAuth-compatible integrations where required
-

CHAPTER 39 — AUTHORIZATION

Every API endpoint shall enforce authorization.

Authorization may depend on:

- User role
- Permission
- Branch

- Department
- Ownership
- Resource relationship
- Organizational scope

Never rely solely on frontend controls.

CHAPTER 40 — API VERSIONING

APIs shall be versioned.

Example:

```
/api/v1/  
/api/v2/
```

Breaking changes shall require a new API version.

Existing supported clients shall not be broken unexpectedly.

CHAPTER 41 — ERROR HANDLING

APIs shall return consistent error structures.

Example conceptual response:

```
{  
  "success": false,  
  "message": "The requested operation could not be completed.",  
  "errors": {},  
  "code": "VALIDATION_ERROR",  
  "request_id": "..."  
}
```

Internal exception details shall never be exposed to production clients.

CHAPTER 42 — PAGINATION

Collection endpoints shall support pagination.

The API shall provide:

- Current page
- Per-page count
- Total count where practical
- Last page
- Navigation metadata

Large datasets shall never be returned without pagination unless explicitly justified.

CHAPTER 43 — FILTERING

APIs shall support standardized filtering.

Examples:

```
?status=available
?property_type=apartment
?branch_id=10
?min_price=100000000
?max_price=500000000
```

Sorting and search parameters shall follow consistent conventions.

CHAPTER 44 — WEBHOOKS

RREP shall support outbound webhooks for selected business events.

Examples:

- Sale completed
- Payment received
- Reservation confirmed
- Customer created
- Property published

Webhook infrastructure shall include:

- Signing
 - Retry
 - Delivery status
 - Failure handling
 - Secret management
-

PART F — FRONTEND STANDARDS

CHAPTER 45 — UI ARCHITECTURE

The web interface shall follow a reusable component architecture.

Major layout components include:

- Sidebar
 - Header
 - Breadcrumbs
 - Content area
 - Footer
 - Notifications
 - Modals
-

CHAPTER 46 — COMPONENT STANDARDS

Reusable components shall be preferred over duplicated markup.

Components shall maintain:

- Consistent naming
 - Consistent spacing
 - Accessibility
 - Responsive behavior
 - Reusable JavaScript behavior
-

CHAPTER 47 — DASHBOARD STANDARDS

Dashboards shall present:

- KPIs
- Trends
- Alerts
- Recent activities
- Charts
- Action shortcuts

Dashboards shall avoid excessive visual complexity.

Role-specific dashboards shall display information relevant to the user's responsibilities.

CHAPTER 48 — FORMS

Forms shall provide:

- Clear labels
 - Validation
 - Helpful error messages
 - Required field indicators
 - Consistent input controls
 - AJAX validation where appropriate
 - Protection against duplicate submissions
-

CHAPTER 49 — TABLES

DataTables or equivalent table components shall support:

- Pagination
- Search
- Sorting
- Filtering
- Export where authorized
- Responsive display
- Bulk operations where appropriate

Large tables shall use server-side processing.

CHAPTER 50 — RESPONSIVE DESIGN

Interfaces shall support:

- Desktop
- Laptop
- Tablet
- Mobile

Critical workflows shall remain usable on smaller screens.

PART G — SECURITY

CHAPTER 51 — IDENTITY MANAGEMENT

Identity management shall support:

- Secure authentication
- Password hashing
- Password reset
- Session protection
- MFA
- Account lockout/rate limiting
- Device/session management

CHAPTER 52 — RBAC

Role-Based Access Control shall be implemented centrally.

Permission examples:

```
properties.view
properties.create
properties.update
properties.delete
sales.view
sales.create
finance.payments.approve
reports.export
users.manage
```

Permissions shall be granular enough to support feature-based licensing and organizational control.

CHAPTER 53 — ENCRYPTION

Data shall be protected:

In Transit

HTTPS/TLS shall be mandatory for production deployments.

At Rest

Sensitive data and backups shall use encryption where supported.

Passwords shall always be securely hashed and never reversibly encrypted.

CHAPTER 54 — SECRETS MANAGEMENT

Secrets shall never be committed to source control.

Examples:

- Database passwords
- API keys
- Payment credentials
- SMTP credentials
- AI provider keys
- Encryption keys

Production secrets shall be stored through environment variables or an appropriate secrets-management system.

CHAPTER 55 — SECURITY MONITORING

Security monitoring shall cover:

- Failed logins
 - Privileged actions
 - Permission changes
 - Suspicious API requests
 - Unusual access patterns
 - Configuration changes
 - Security exceptions
-

CHAPTER 56 — OWASP COMPLIANCE

Development shall consider applicable OWASP risks, including:

- Broken access control
- Injection
- Authentication failures
- Cryptographic failures
- Security misconfiguration
- Vulnerable dependencies
- Logging failures
- SSRF
- XSS
- CSRF

Security testing shall be incorporated into the development lifecycle.

PART H — DEVOPS

CHAPTER 57 — GIT STRATEGY

Git shall be used for source control.

Recommended branch structure:

```
main
develop
feature/*
bugfix/*
hotfix/*
release/*
```

The exact branching model may be simplified depending on team size.

All production changes shall be traceable to commits and reviewed appropriately.

CHAPTER 58 — DOCKER

Containerization shall be supported for development and standardized deployments.

Potential containers include:

```
app
nginx
mysql
redis
worker
```

scheduler
search

Docker shall not be mandatory for every deployment if customer infrastructure requires native installation.

CHAPTER 59 — CI/CD

The project shall implement automated pipelines for:

1. Code checkout
2. Dependency installation
3. Static analysis
4. Automated tests
5. Build
6. Security checks
7. Deployment

Production deployment shall require controlled release procedures.

CHAPTER 60 — MONITORING

Production installations shall monitor:

- CPU
- Memory
- Disk
- Database
- Queue
- Application health
- API response time
- Error rates
- Storage
- External integrations

Health-check endpoints shall be provided.

CHAPTER 61 — LOGGING

Application logs shall be structured and categorized.

Examples:

- Application
- Authentication
- API
- Payment
- Integration
- AI
- Queue
- Security

Logs shall not contain passwords, secrets, full payment credentials, or unnecessary sensitive personal information.

CHAPTER 62 — BACKUP

Backup strategy shall cover:

- Database
- Uploaded documents
- Configuration
- Encryption keys where recoverable
- Critical deployment information

Backups shall be:

- Automated
- Retained according to policy
- Monitored
- Tested through restoration procedures

CHAPTER 63 — DISASTER RECOVERY

Each production deployment shall define:

- Recovery Point Objective (RPO)
- Recovery Time Objective (RTO)
- Backup frequency
- Recovery procedure
- Failover procedure
- Responsible personnel

Disaster recovery shall be tested periodically.

PART I — QUALITY

CHAPTER 64 — CODING STANDARDS

The engineering team shall follow:

- PSR standards
- Laravel conventions
- Consistent naming
- SOLID principles
- DRY principles
- Clear separation of concerns

Code shall prioritize readability and maintainability.

CHAPTER 65 — TESTING STRATEGY

Testing shall occur at multiple levels.

Unit Tests

Test individual classes and business rules.

Feature Tests

Test complete application features.

Integration Tests

Test:

- APIs
- Payments
- Messaging
- External services
- Storage

Browser Tests

Test critical user journeys.

CHAPTER 66 — PERFORMANCE TESTING

Performance testing shall evaluate:

- Concurrent users
- API response times
- Database queries
- Large datasets
- File uploads
- Report generation
- Background processing

Critical endpoints shall have defined performance targets.

CHAPTER 67 — SECURITY TESTING

Security testing shall include:

- Authentication testing
 - Authorization testing
 - Input validation
 - Injection testing
 - File upload testing
 - API security
 - Dependency scanning
 - Vulnerability testing
 - Penetration testing before major enterprise releases
-

CHAPTER 68 — DOCUMENTATION STANDARDS

Technical documentation shall include:

- Installation
- Configuration
- APIs
- Database
- Modules

- Deployment
- Troubleshooting
- Security
- Integrations

API documentation shall be machine-readable where possible using OpenAPI-compatible documentation.

PART J — APPENDICES

CHAPTER 69 — RECOMMENDED FOLDER STRUCTURE

A recommended high-level structure is:

```
rrep/
├── app/
│   ├── Http/
│   ├── Models/
│   ├── Modules/
│   │   ├── Administration/
│   │   ├── Property/
│   │   ├── CRM/
│   │   ├── Sales/
│   │   ├── Finance/
│   │   ├── Survey/
│   │   ├── Marketing/
│   │   ├── Documents/
│   │   ├── Workflow/
│   │   ├── Reporting/
│   │   ├── Integrations/
│   │   └── AI/
│   ├── Services/
│   └── Support/
├── config/
├── database/
│   ├── migrations/
│   ├── seeders/
│   └── factories/
├── resources/
│   ├── views/
│   ├── js/
│   └── css/
├── routes/
│   ├── web.php
│   ├── api.php
│   └── channels.php
```

```
|
|— storage/
|— tests/
|— docker/
|— documentation/
```

This structure may be refined during implementation.

CHAPTER 70 — NAMING STANDARDS

Classes

Use PascalCase:

```
PropertyService
CustomerController
PaymentProcessor
```

Methods

Use camelCase:

```
createProperty()
calculateCommission()
approveReservation()
```

Variables

Use camelCase:

```
$propertyId
$customerName
$totalAmount
```

Database

Use snake_case:

```
property_id
customer_id
payment_transactions
```

Routes

Use REST conventions:

```
/properties
/properties/{property}
```

/customers
/sales
/payments

CHAPTER 71 — DESIGN PATTERNS

The following patterns may be used where appropriate:

Service Pattern

For reusable business operations.

Strategy Pattern

For interchangeable algorithms.

Examples:

- Commission calculation
- Pricing
- Payment providers
- AI providers

Factory Pattern

For creating provider-specific implementations.

Observer/Event Pattern

For reacting to business events.

Adapter Pattern

For external integrations.

Repository Pattern

For complex or abstracted persistence.

DTO Pattern

For transferring structured data between layers.

Policy Pattern

For authorization.

Specification Pattern

For complex filtering and business criteria where justified.

Patterns shall be used to solve real problems, not to increase architectural complexity unnecessarily.

CHAPTER 72 — FUTURE ARCHITECTURE

72.1 Architectural Evolution

The initial RREP architecture shall be implemented as a well-structured modular monolith unless measured requirements justify decomposition.

This is an important architectural decision.

The platform should NOT begin as a collection of dozens of microservices.

A modular monolith provides:

- Lower operational complexity
- Easier development
- Easier deployment
- Strong transactional consistency
- Lower infrastructure cost
- Faster product iteration

However, module boundaries shall be designed so that high-value components can later be extracted into independent services.

72.2 Potential Future Services

As scale increases, the following components may become independently deployable:

Identity Service
Notification Service
Search Service
File Service
Reporting Service
Integration Service
AI Service
Payment Service
GIS Service

Extraction shall be based on measurable technical or business requirements.

72.3 Event-Driven Evolution

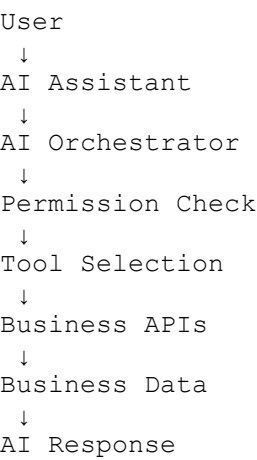
The platform shall gradually increase its use of events for:

- Notifications
- Analytics
- Integrations
- AI processing
- Search indexing
- Audit processing

This allows modules to become increasingly decoupled.

72.4 AI Evolution

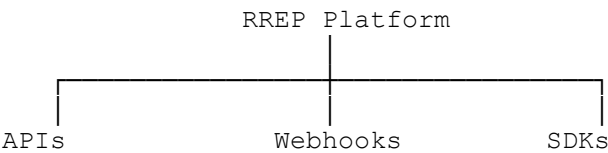
The AI architecture shall eventually support:

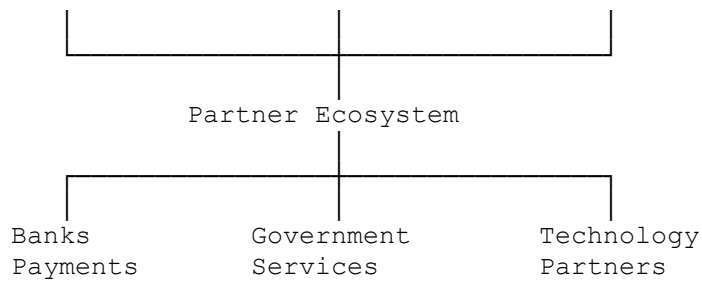


AI agents shall never bypass normal authorization mechanisms.

72.5 Ecosystem Architecture

Future versions may provide:





72.6 Plugin Architecture

The long-term architecture should permit selected capabilities to be installed as extensions.

Potential extensions include:

- Country-specific tax modules
- Payment integrations
- Government integrations
- AI providers
- GIS providers
- Industry-specific modules
- Reporting packs

Extensions shall use documented interfaces and shall not directly modify the platform core.

72.7 Feature Licensing Architecture

Because RREP is a reusable commercial product, licensing shall be implemented as a first-class platform capability.

The platform shall be able to determine:

```
License
↓
Edition
↓
Enabled Modules
↓
Enabled Features
↓
Usage Limits
↓
User Permissions
```

Licensing must be separated from authorization.

For example:

- A user may have permission to access Finance.
- But if the customer's license does not include Finance, the module remains unavailable.

Both checks must succeed.

72.8 License Architecture

A deployment shall maintain a license record containing appropriate information such as:

- License identifier
- Customer identifier
- Product edition
- Licensed modules
- Enabled features
- User limits
- Expiry date
- License status
- Activation status
- Signature/verification data

Sensitive licensing mechanisms shall not expose proprietary signing secrets to the client.

72.9 Offline/Local Operation

Where a customer deployment requires local operation, the platform shall continue functioning without dependence on RehoSpace infrastructure except for features that explicitly require external services.

Examples:

Core operations may remain available locally:

- Property management
- CRM
- Sales
- Finance
- Reporting
- User management

External-dependent capabilities may become unavailable or degraded:

- SMS
- WhatsApp
- Cloud AI

- External payment gateways
- External maps

The platform shall clearly indicate integration availability.

72.10 Data Portability

Customers shall be able to export their business data according to their license and contractual terms.

Supported formats may include:

- CSV
- Excel
- JSON
- PDF
- Database backup

Export mechanisms shall respect authorization and privacy requirements.

72.11 Localization

The architecture shall support:

- Multiple languages
- Multiple currencies
- Date formats
- Number formats
- Time zones
- Country-specific configuration
- Tax rules
- Address structures

The first deployment may prioritize Tanzania, while the architecture remains capable of supporting additional countries.

72.12 Observability

Future architecture shall support unified observability through:

- Metrics
- Logs

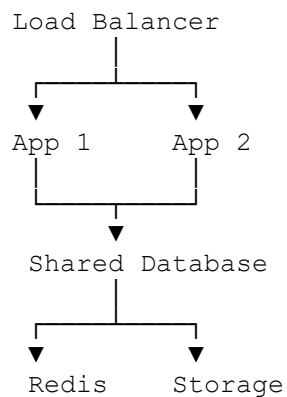
- Traces
- Health checks
- Alerts

Correlation IDs shall allow engineers to trace a request across:

```
User Request
→ API
→ Business Service
→ Database
→ Queue
→ External Provider
```

72.13 High Availability

Enterprise deployments may eventually support:



High availability shall be an enterprise deployment capability rather than a mandatory requirement for every small installation.

72.14 Scalability Strategy

The platform shall scale progressively.

Stage 1

Single application server:

```
RREP
+
MySQL
+
Redis
```

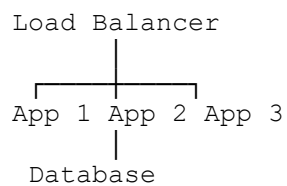
Stage 2

Separate workers:

Web
Worker
Scheduler
MySQL
Redis
Storage

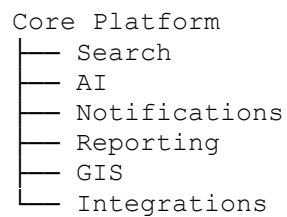
Stage 3

Multiple application servers:



Stage 4

Specialized services:



Scaling decisions shall be driven by actual workload and business requirements.

TECHNICAL GOVERNANCE

Architecture Decision Records

Major technical decisions shall be documented using Architecture Decision Records (ADRs).

Each ADR should document:

- Decision
- Context
- Alternatives
- Selected approach
- Reason
- Consequences

Examples:

- Why modular monolith was selected.
 - Why MySQL was selected.
 - Why Redis was selected.
 - Why a particular GIS provider was selected.
 - Why an AI provider was selected.
-

ENGINEERING CHANGE CONTROL

Changes affecting architecture shall be reviewed by the technical lead or designated architecture authority.

Changes requiring architectural review include:

- Database engine changes
 - Authentication changes
 - Major framework changes
 - Module boundary changes
 - New external infrastructure
 - New payment integrations
 - Major AI architecture changes
 - Security architecture changes
 - Deployment architecture changes
-

DEVELOPMENT ENVIRONMENT STANDARD

Each developer environment shall provide:

- PHP
- Composer
- Node.js/npm where required
- MySQL
- Redis where required
- Git
- Required PHP extensions
- Required frontend tooling

Docker may be provided to standardize onboarding.

PRODUCTION ENVIRONMENT BASELINE

A standard production deployment shall include:

Linux
Nginx
PHP-FPM
Laravel
MySQL
Redis
Queue Worker
Scheduler
SSL/TLS
File Storage
Monitoring
Backup

Exact infrastructure requirements shall depend on deployment scale.

ENGINEERING RELEASE PRINCIPLE

No feature shall be considered production-ready until:

1. Functional requirements are satisfied.
 2. Authorization is implemented.
 3. Validation is implemented.
 4. Audit requirements are addressed.
 5. Automated tests exist for critical behavior.
 6. Error handling is implemented.
 7. Logging is appropriate.
 8. Performance is acceptable.
 9. Security review is completed where applicable.
 10. Documentation is updated.
-

FINAL TECHNICAL PRINCIPLE

The most important architectural rule for RREP is:

Build a reusable platform, not a one-client application.

Every major engineering decision shall therefore be evaluated against five questions:

1. Can this capability be reused by another RREP customer?

2. Can it be configured instead of hard-coded?
3. Can it evolve without breaking existing deployments?
4. Can it be exposed through an API where appropriate?
5. Can it be maintained independently of customer-specific requirements?

If the answer is consistently yes, the architecture remains aligned with the commercial product strategy established in Volume 0 and the functional requirements established in Volume I.

END OF VOLUME II

RehoSpace Real Estate Platform (RREP)
Technical Architecture & Engineering Specification
Version 1.0
